EcoLab1

Создание компонента, реализующего алгоритм Быстрой сортировки

1. Алгоритм
2. Оценка сложности
3. Улучшения
4. Результаты тестирования
5. Поддержка других компонентов
6. Визуализация

Выполнила

Самсонова Карина

20ПИ-1

1. Алгоритм

Быстрая сортировка – один из самых известных и широко используемых алгоритмов сортировки. Среднее время работы $O(n*log\ n)$, что является асимптотически оптимальным временем работы для алгоритма, основанного на сравнении. Хотя время работы алгоритма для массива из n элементов в худшем случае может составить $O(n^2)$, на практике этот алгоритм является одним из самых быстрых.

Быстрый метод сортировки функционирует по принципу «разделяй и властвуй».

- Массив $a[l…r]$ типа T разбивается на два (возможно пустых) подмассива $a[l…q]$ и $a[q+1…r]$, таких, что каждый элемент $a[l…q]$ меньше или равен $a[q]$, который в свою очередь, не превышает любой элемент подмассива $a[q+1…r]$. Индекс вычисляется в ходе процедуры разбиения.

- Подмассивы $a[l…q]$ и $a[q+1…r]$ сортируются в помощью рекурсивного вызова процедуры быстрой сортировки.

- Поскольку подмассивы сортируются на месте, для их объединения не требуются никакие действия: весь массив $a[l…r]$ оказывается отсортированным.

Псевдокод

```
void quicksort(a: T[n], int l, int r)
     if l < r
        int q = partition(a, l, r)
        quicksort(a, l, q)
        quicksort(a, q + 1, r)
```

2. Оценка сложности

Среднее.

Ясно, что операция разделения массива на две части относительно опорного элемента занимает время $O(n)$. Поскольку все операции разделения, проделываемые на одной глубине рекурсии, обрабатывают разные части исходного массива, размер которого постоянен, суммарно на каждом уровне рекурсии потребуется также $O(n)$ операций. Следовательно, общая сложность алгоритма определяется лишь количеством разделений, то есть глубиной рекурсии. Глубина рекурсии, в свою очередь, зависит от сочетания входных данных и способа определения опорного элемента.

Среднюю сложность при случайном распределении входных данных можно оценить лишь вероятностно.

Прежде всего необходимо заметить, что в действительности необязательно, чтобы опорный элемент всякий раз делил массив на две одинаковых части. Например, если на каждом этапе будет происходить разделение на массивы длиной 75% и 25% от исходного, глубина рекурсии будет равна $log_{4/3} n$, а это по-прежнему даёт сложность $O(n*log\ n)$. Вообще, при любом фиксированном соотношении между левой и правой частями разделения сложность алгоритма будет той же, только с разными константами.

Будем считать «удачным» разделением такое, при котором опорный элемент окажется среди центральных 50% элементов разделяемой части массива; ясно, вероятность удачи при случайном распределении элементов составляет 0,5. При удачном разделении размеры выделенных подмассивов составят не менее 25% и не более 75 % от исходного. Поскольку каждый выделенный подмассив также будет иметь случайное распределение, все эти рассуждения применимы к любому этапу сортировки и любому исходному фрагменту массива.

Удачное разделение даёт глубину рекурсии не более $log_{4/3}\ n$. Поскольку вероятность удачи равна 0,5, для получения k удачных разделений в среднем потребуется $2*k$ рекурсивных вызовов, чтобы опорный элемент k раз оказался среди центральных 50% массива.

Применяя эти соображения, можно заключить, что в среднем глубина рекурсии не превысит $2*\log_{4/3} n$, что равно $O(\log n)$. А поскольку на каждом уровне рекурсии по-прежнему выполняется не более $O(n)$ операций, средняя сложность составит $O(n*\log n)$.

Худший случай.

В самом несбалансированном варианте каждое разделение даёт два подмассива размерами 1 и n-1, то есть при каждом рекурсивном вызове больший массив будет на 1 короче, чем в предыдущий раз. Такое может произойти, если в качестве опорного на каждом этапе будет выбран элемент либо наименьший, либо наибольший из всех обрабатываемых. При простейшем выборе опорного элемента — первого или последнего в массиве, — такой эффект даст уже отсортированный (в прямом или обратном порядке) массив, для среднего или любого другого фиксированного элемента «массив худшего случая» также может быть специально подобран. В этом случае потребуется n-1 операций разделения, а общее время работы составит

$\sum^{n}_{i=0}(n - i) = O(n^2)$ операций, то есть сортировка будет выполняться за квадратичное время. Но количество обменов и, соответственно, время работы — это не самый большой его недостаток. Хуже то, что в таком случае глубина рекурсии при выполнении алгоритма достигнет n, что будет означать n-кратное сохранение адреса возврата и локальных переменных процедуры разделения массивов. Для больших значений n худший случай может привести к исчерпанию памяти (переполнению стека) во время работы программы.

3. Улучшения

Разбиение Хоара.

Оригинальная схема разбиения, описанная Тони Хоаром, использует два указателя, которые движутся навстречу друг другу с обоих концов разбитого массива, пока не обнаружат инверсию: пару элементов, где элемент под первым указателем больше, чем pivot, а элемент под вторым указателем - меньше; если в этот момент первый указатель все еще находится перед вторым, эти элементы находятся в неправильном порядке относительно друг друга, и затем они обмениваются. После этого указатели перемещаются внутрь, и поиск инверсии повторяется; когда в конечном итоге указатели пересекаются, обмен не выполняется; найдено допустимое разделение с точкой разделения между пересекающимися указателями(любые записи, которые могут быть строго между скрещенными указателями равны оси поворота и могут быть исключены из обоих сформированных поддиапазонов).

Схема Хоара эффективнее схемы Ломуто, так как происходит в среднем в три раза меньше обменов (swap) элементов, и разбиение эффективнее, даже когда все элементы равны. Подобно схеме Ломуто, данная схема также показывает эффективность в $O(n^2)$, когда входной массив уже отсортирован. Сортировка с использованием данной схемы нестабильна. Следует заметить, что конечная позиция опорного элемента необязательно совпадает с возвращённым индексом.

Медиана из трех.

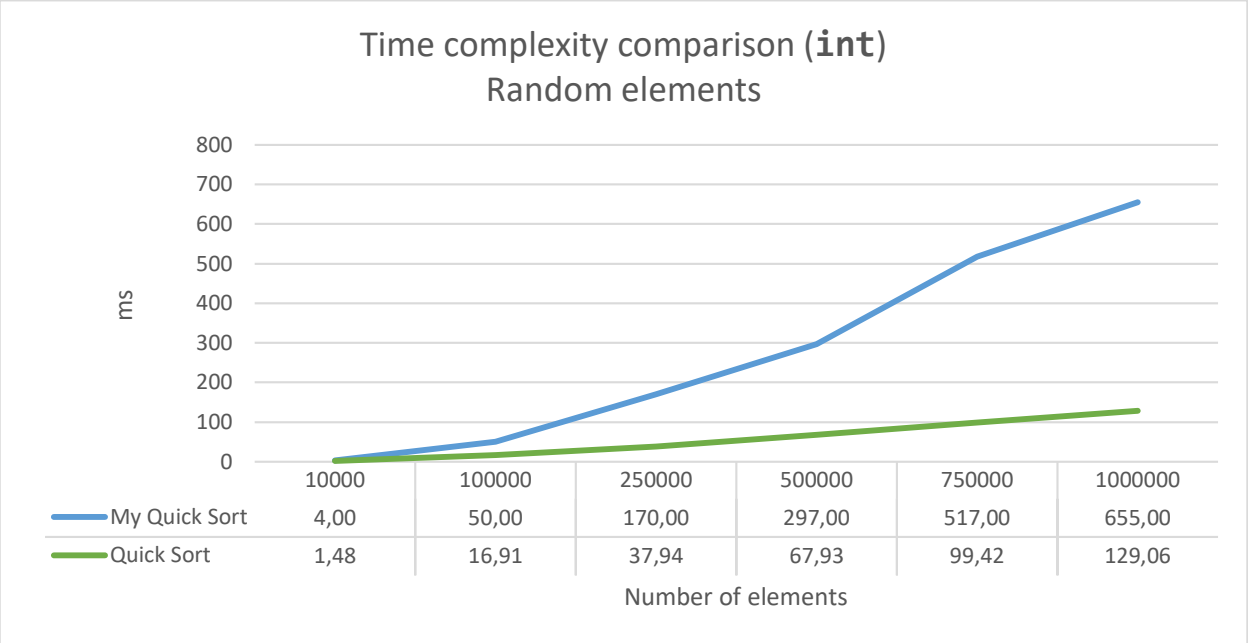
Другим усовершенствованием быстрой сортировки является использование центрального элемента, который с большей вероятностью делит файл близко к середине. Распространенный способ определения лучшего центрального элемента заключается в выборке трех элементов из файла и использовании в качестве центрального элемента их медианы. Если выбрать за эти три элемента левый, правый и средний элементы массива, то заодно можно включить в схему и сигнальные элементы: сортируем три выбранных элемента, потом обмениваем средний из них с элементом *a[r-1]*, и затем выполняем алгоритм разбиения для *a[l+1], ..., a[r-2]*.

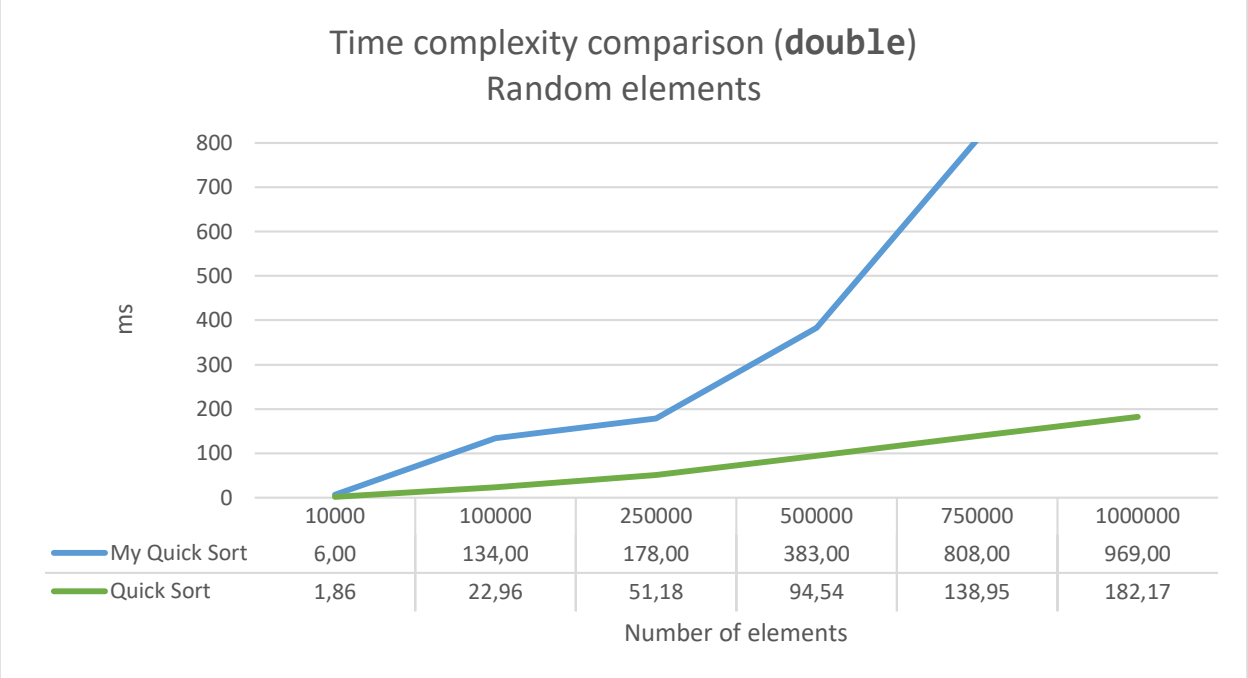
Это усовершенствование называется методом медианы из трех (median-of-three).

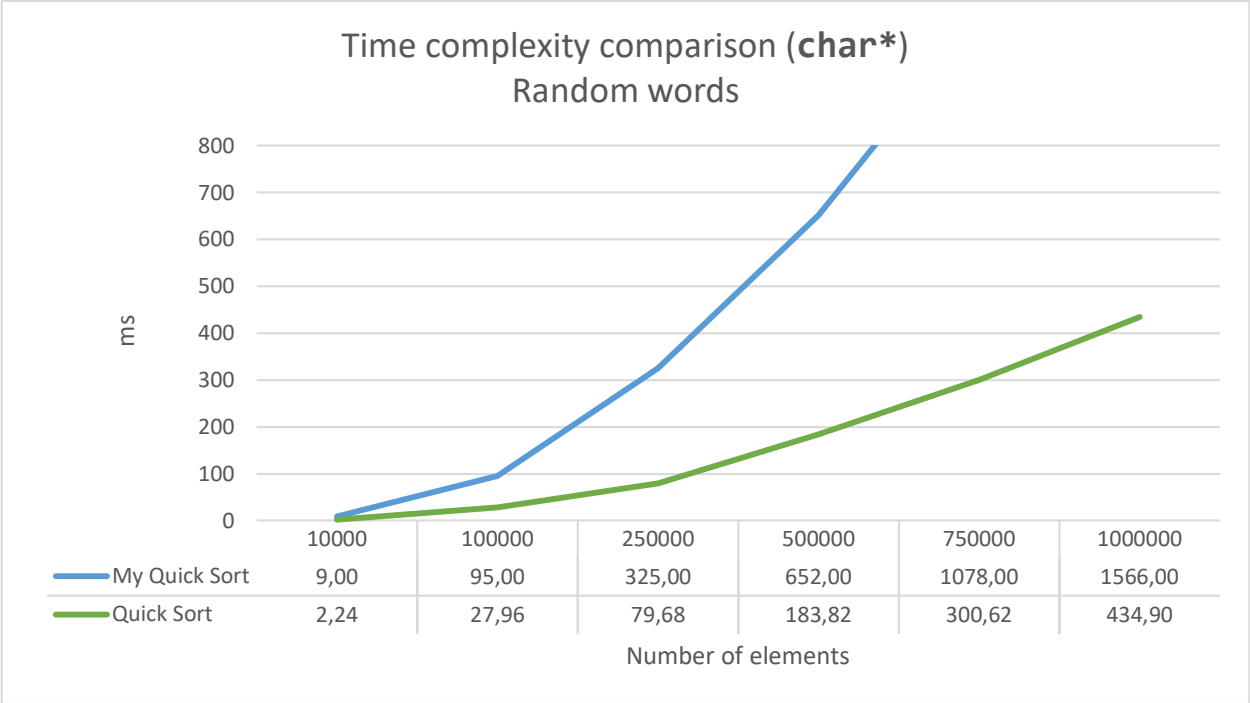
Метод медианы из трех повышает эффективность сортировки тремя способами. Во-первых, он существенно снижает вероятность появления наихудшего случая при сортировке любого реального файла. Чтобы сортировка выполнилась за время порядка n^2, два из трех выбранных элементов должны быть наибольшими или наименьшими элементами в файле, и это событие должно постоянно повторяться для большинства разбиений. Во-вторых, он устраняет необходимость в сигнальном элементе, т.к. эту функцию берет на себя один из трех элементов, проверенных перед разбиением. В-третьих, он уменьшает общее среднее время выполнения алгоритма примерно на 5%.

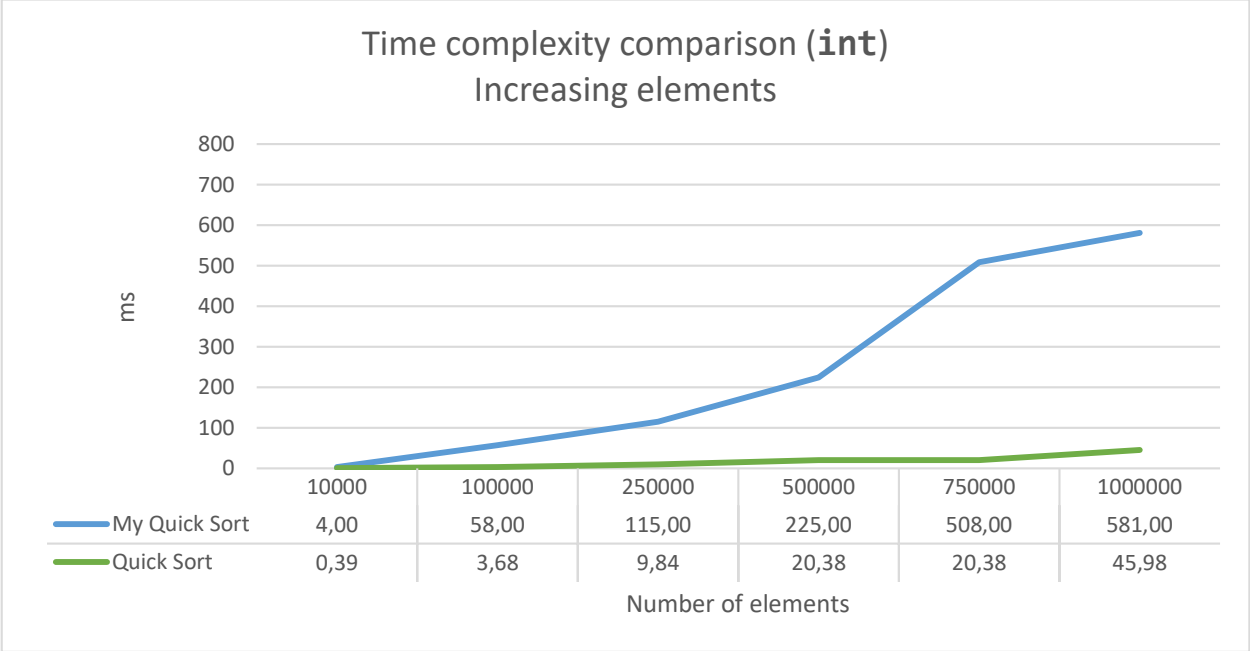
4. Результаты тестирования

Было проведено тестирование программы с различными входными данными(рандомный int, рандомный double, рандомный char*, возрастающий int, а также числа элементов от 10000 до 1000000). Результаты тестирования можно увидеть на графиках ниже:









Можно отметить, что время выполнения сортировки напрямую зависит от входных данных. В целом, моя быстрая сортировка хоть и уступает по скорости стандартной qsort, но показывает хорошие результаты даже на крупных массивах данных. Наихудшие результаты были получены на массивах типа double и char. Если пытаться понять причину, по которой моя сортировка уступает стандарту, можно предположить, что в этом виновата разница реализаций.

5. Поддержка других компонентов

Задача второй лабораторной работы: «в ранее созданный компонент используя приемы программирования включение/агрегирование добавить компонент(-ы) калькулятора, выполняющий(-ие) операции сложения, вычитания, деления и умножения. Продемонстрировать свойства интерфейсов, показать, что через любой указатель на интерфейс можно получить указатель на любой другой запрашиваемый интерфейс используя метод QueryInterface». Демонстрационными компонентами были выбраны компоненты A, B, D и E.

Для поддержки сторонних компонентов в структуру CEcoLab1 были добавлены указатели на новые интерфейсы и их виртуальные таблицы:

```
IEcoCalculatorXVTbl* m_pVTblIEcoCalculatorX;

IEcoCalculatorYVTbl* m_pVTblIEcoCalculatorY;

IEcoCalculatorX* m_pIEcoCalculatorX;

IEcoCalculatorY* m_pIEcoCalculatorY;

IEcoUnknown* m_pInnerUnknown;
```

Пример метода включения и взаимозаменяемости компонентов (получаем компонент D, если не удалось, получаем компонент E):

```
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorD, 0,
&IID_IEcoCalculatorY, (void**) &pCMe->m_pIEcoCalculatorY);
if (result != 0 || pCMe->m_pIEcoCalculatorY == 0) {
       result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorE, 0,
&IID_IEcoCalculatorY, (void**) &pCMe->m_pIEcoCalculatorY);
}
```

Теперь, если компонент D/E с интерфейсов IEcoCalculatorX доступен, мы можем имплементировать методы интерфейса внутри нашего компонента и определить их в виртуальную таблицу, которая будет присвоена соответствующей переменной во время исполнения create функции. Для передачи интерфейса пользователю добавим else if в метод CEcoLab1_QueryInterface.

```
else if (IsEqualUGUID(riid, &IID_IEcoCalculatorX)) {
    *ppv = &pCMe->m_pVTblIEcoCalculatorX;
    pCMe->m_pVTblIEcoLab1->AddRef((IEcoLab1*) pCMe);
}
```

Для реализации метода агрегирования в структуре CEcoLab1 был создан указатель m_pInnerUnknown. Он имеет базовый для интерфейсов тип IEcoUnknown, реализующий метод QueryInterface, который нужен для передачи указателя пользователю.

Вот пример получения интерфейса методом агрегирования. Можно видеть, что, если интерфейс не удается получить из компонента B, то он будет получен из компонента A методом включения.

```
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorB, pOuterUnknown,
&IID_IEcoUnknown, (void**) &pCMe->m_pInnerUnknown);
if (result != 0 || pCMe->m_pInnerUnknown == 0) {
    result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorA, 0,
&IID_IEcoCalculatorX, (void**) &pCMe->m_pIEcoCalculatorX);
}
```

Пример запроса компонента:

Метод CEcoLab1_QueryInterface используется внутри нашего компонента с такой же модификацией как в случае включения, то есть, else if, где, при условии, что пользователь запрашивает поддерживаемый компонент и указатель на него не равен 0, происходит вызов метода QueryInterface у полученного компонента по указателю m_pInnerUnknown. Пример:

```
else if (IsEqualUGUID(riid, &IID_IEcoCalculatorX) && pCMe->m_pInnerUnknown != 0) {
    return pCMe->m_pInnerUnknown->pVTbl->QueryInterface(pCMe->m_pInnerUnknown, riid,
ppv);
```

Теперь компонент поддерживает агрегирование, включение и взаимозаменяемость компонентов. Юнит-тесты наглядно демонстрируют работу программы.

6. Визуализация

Для визуализации работы компонента было создано несколько обратных вызовов:

- OnPrintStringCallback выводит строку to_print:

```
int16_t(ECOCALLMETHOD* OnPrintStringCallback)(/* in */ struct IEcoLab1Events* me,
char_t* to_print);
```

- OnSubarrayToSortCallback выводит массив, выделяя белым сортируемый подмассив от lower до upper:

```
int16_t(ECOCALLMETHOD* OnSubarrayToSortCallback)(/* in */ struct IEcoLab1Events* me,
void* arr, int num, int size, int lower, int upper);
```

- OnMedianIsReadyCallback выводит массив, выделяя зеленым крайние элементы подмассива и медиану 3-х:

```
int16_t (ECOCALLMETHOD* OnMedianIsReadyCallback)(/* in */ struct IEcoLab1Events* me,
void* arr, int num, int size, int lower, int mid, int upper);
```

- OnPartitionIterationCallback визуализирует итерацию сортировки, выделяя i - синим, j - зеленым, pivot - желтым, переставляемые элементы - красным:

```
int16_t(ECOCALLMETHOD* OnPartitionIterationCallback)(/* in */ struct IEcoLab1Events*
me, void* arr, int num, int size, int lower, int upper, int i, int j, int is_swapped,
int pivot);
```

Для удобства наблюдения за визуалом каждый callback заканчивается функцией Sleep(), выжидая определенное время (в данном случае 3 секунды) после вывода информации.